

Tawsileh Intercity

Prototype Technical & Product Documentation

Project Name: Tawsileh Intercity

Arabic Name: "تطبيق"توصيلة بين المحافظات"

Prototype Route: Jerash ↔ Amman

Platforms: Android & iOS

Document Type: Product Requirements, UI/UX, Frontend, Backend, Database, API & Matching Engine Specification

Version: 1.0

Status: Prototype / MVP Specification

1. Project Overview

1.1 Introduction

Tawsileh Intercity is a ride-sharing and passenger aggregation mobile application designed specifically for intercity transportation in Jordan.

Unlike conventional ride-hailing applications, where one passenger typically pays the full trip cost, Tawsileh allows multiple passengers traveling along compatible routes to share the same vehicle and divide the trip cost.

The prototype will initially focus exclusively on the:

Jerash ↔ Amman

route.

The system will match passengers with captains whose predefined route is compatible with the passenger's pickup location and destination.

The initial vehicle capacity supported by the system is:

Maximum: 4 passengers per trip.

The fundamental objective is to reduce the individual transportation cost for passengers while allowing captains to serve multiple passengers during the same intercity journey.

2. Problem Statement

Intercity transportation in Jordan can be relatively expensive when a passenger requires a private ride.

For example:

A private trip from Jerash to Amman may cost approximately:

12 JOD

A single passenger therefore pays the entire amount.

With Tawsileh:

Passengers	Example Cost Per Passenger
1	12 JOD
2	6 JOD
3	4 JOD
4	3 JOD

The exact pricing model may be modified later to account for route deviations, passenger pickup locations, and operational costs.

The prototype will prioritize validating the **matching model**, not the final commercial pricing model.

3. Product Vision

Tawsileh aims to become an intercity shared transportation platform connecting passengers traveling between Jordanian cities.

The long-term vision is:

Make intercity transportation more affordable by intelligently combining passengers traveling along compatible routes.

The system should eventually support:

- Jerash ↔ Amman
- Irbid ↔ Amman
- Zarqa ↔ Amman
- Ajloun ↔ Amman
- Salt ↔ Amman
- Madaba ↔ Amman
- Other intercity routes

However, the prototype must only implement:

Jerash ↔ Amman

4. Prototype Objectives

The prototype must prove that the following process works:

1. Passenger opens the application.
2. Passenger selects pickup location.
3. Passenger selects destination.
4. Passenger submits a ride request.
5. System identifies compatible captain routes.
6. System evaluates route compatibility.
7. System checks pickup/destination proximity.
8. System checks available seats.

9. System ranks compatible captains.
10. Passenger receives available matches.
11. Captain receives the ride request.
12. Captain accepts the passenger.
13. Passenger sees captain information and location.
14. Captain picks up passenger.
15. Trip begins.
16. Trip is completed.
17. Available seats are updated.

5. User Roles

The prototype contains three major user roles.

5.1 Passenger

A passenger requests transportation between cities.

Capabilities:

- Create account
- Log in
- Set pickup location
- Set destination
- Request a ride
- View matching captains
- View estimated price
- Confirm ride
- Cancel ride
- Track captain
- View trip status
- View trip history
- Rate captain

5.2 Captain

A captain provides transportation.

Capabilities:

- Create captain account
- Submit basic verification information
- Select route
- Set route direction
- Set pickup radius
- Set available seats
- Go online/offline
- Receive passenger requests
- Accept/reject requests
- View passenger information
- Navigate to passenger
- Start trip
- Complete trip
- View trip history

5.3 Administrator

The administrator manages the platform.

Capabilities:

- View users
- View captains
- Approve/reject captains
- View active trips
- Monitor active captains
- Manage routes
- Manage complaints
- Suspend accounts

- View system statistics

The admin dashboard is not required to be a full production system in the first prototype, but the backend architecture should support it.

6. MVP Scope

Included

The first prototype includes:

- Android application
- iOS application
- Passenger mode
- Captain mode
- Authentication
- GPS location
- Map interface
- Pickup selection
- Destination selection
- Jerash ↔ Amman routes
- Captain route configuration
- Pickup radius
- Matching algorithm
- Seat availability
- Ride requests
- Captain acceptance
- Real-time trip status
- Captain location updates
- Push notifications
- Trip history
- Basic rating system
- Basic admin dashboard

7. Explicitly Excluded From Prototype

The following should NOT be implemented initially:

- Multiple cities
- Advanced payment gateway
- Wallet
- Subscription system
- Promotions
- Referral system
- Corporate accounts
- Advanced analytics
- AI-based matching
- Dynamic surge pricing
- Loyalty program
- Multi-stop optimization
- Complex driver incentives

These can be added after the core system has been validated.

8. Recommended Technology Stack

8.1 Mobile Frontend

Flutter + Dart

Reason:

- Single codebase
- Android support
- iOS support
- Good map integration
- Fast prototyping
- Strong UI capabilities

- Easy component reuse

9. Backend

Recommended:

Node.js + TypeScript + NestJS

Reasons:

- Modular architecture
- Type safety
- REST API support
- WebSocket support
- Good scalability
- Strong ecosystem
- Clean separation of modules

10. Database

Recommended:

PostgreSQL + PostGIS

PostGIS is particularly important because the application depends heavily on geographic operations.

Examples:

- Distance between two coordinates
- Point-to-route distance
- Radius queries
- Geographic filtering
- Route geometry storage

11. Maps & Location Services

Recommended:

Google Maps Platform

Potential services:

- Maps SDK
- Places API
- Geocoding API
- Routes API
- Distance calculations

The application should abstract map functionality behind a service layer so that another provider can be introduced later if required.

12. Real-Time Communication

Recommended:

WebSockets + Socket.IO

Used for:

- Captain location updates
- Ride request updates
- Ride acceptance
- Ride cancellation
- Trip status
- Seat availability
- Passenger/captain synchronization

13. Notifications

Recommended:

Firestore Cloud Messaging (FCM)

Examples:

- New ride request
- Captain accepted request
- Captain approaching
- Trip started
- Trip completed
- Ride cancelled

14. Admin Dashboard

Recommended:

React + Next.js + TypeScript

The dashboard communicates with the same backend API.

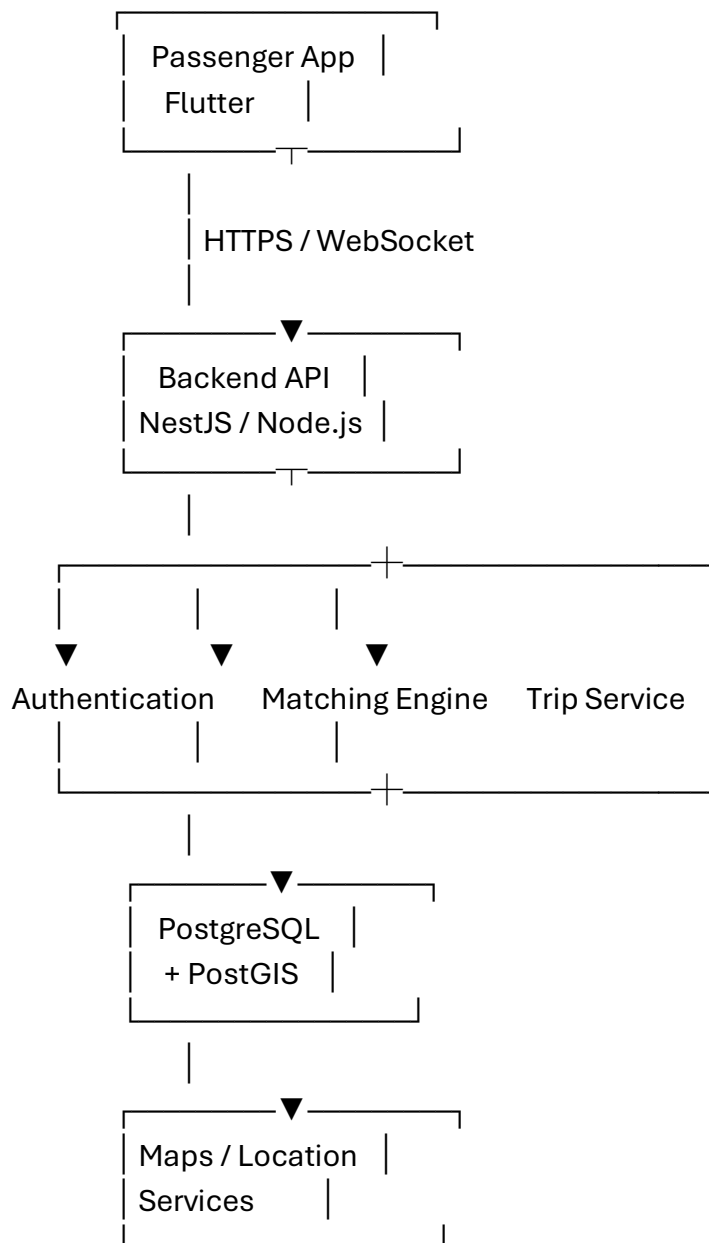
15. Development Tools

Recommended development environment:

- Visual Studio Code
- Android Studio
- Xcode
- Git
- GitHub
- Postman

- Docker
- Figma
- PostgreSQL
- pgAdmin or another PostgreSQL client

16. High-Level Architecture



Captain App
Flutter

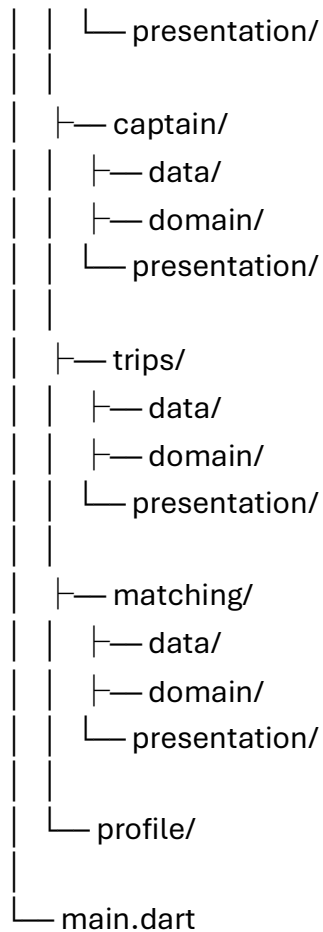
17. Mobile Application Architecture

Recommended architecture:

Clean Architecture + Feature-Based Structure

Example:

```
lib/
|
|— core/
|   |— constants/
|   |— errors/
|   |— network/
|   |— theme/
|   |— utils/
|   └— services/
|
|— features/
|   |
|   |— authentication/
|   |   |— data/
|   |   |— domain/
|   |   └— presentation/
|   |
|   |— passenger/
|   |   |— data/
|   |   └— domain/
```



18. UI/UX Design Principles

The application should follow these principles:

Simplicity

The passenger should be able to request a ride within a few steps.

Map-first design

The map should be the primary visual element.

Clear pricing

The passenger must understand how much they are expected to pay.

Clear trip status

At every stage, the passenger should know:

- What is happening?
- Who is driving?
- Where is the captain?
- What happens next?

Arabic-first consideration

Because the initial market is Jordan, the production application should support Arabic as the primary language.

The prototype should support:

- Arabic
- English

The interface must support RTL properly.

19. Passenger UI Flow

Splash Screen



Onboarding

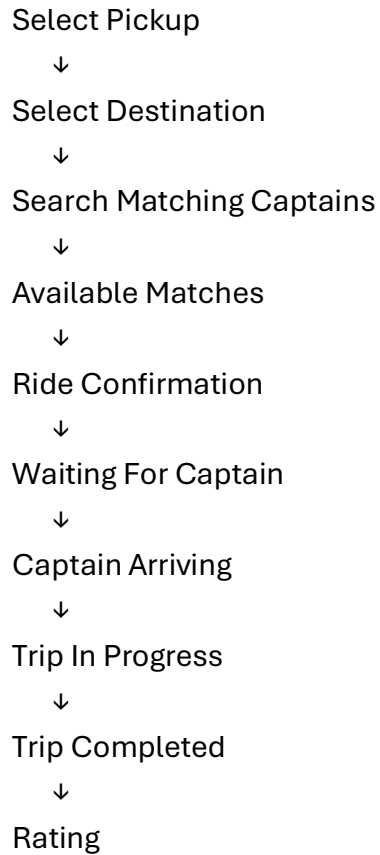


Login / Register



Passenger Home





20. Passenger Screens

20.1 Splash Screen

Contains:

- Tawsileh logo
- Application name
- Loading indicator

Purpose:

Initialize:

- Authentication state

- Location permission state
- Network connection

21. Onboarding

Three optional onboarding screens.

Screen 1

Affordable Intercity Travel

Explain shared rides.

Screen 2

Share Your Ride

Passengers traveling in the same direction share the trip.

Screen 3

Travel Between Cities

Start with Jerash ↔ Amman.

CTA:

Get Started

22. Authentication Screen

Options:

- Phone number
- OTP verification

Potential future options:

- Email
- Google
- Apple

For the prototype, phone + OTP is sufficient.

23. Passenger Home Screen

Main elements:

Tawsileh

Good morning

Where are you going?

[Pickup location]

[Destination]

[Find a Ride]

Recent Trips

Jerash → Amman

Amman → Jerash

The map can occupy the background or upper section.

24. Pickup Selection

The passenger can:

1. Use current location.
2. Search for a location.
3. Move the map.
4. Confirm the location.

The UI should display:

Pickup Location

and:

Confirm Pickup

25. Destination Selection

The passenger selects:

- Destination through search
- Map selection
- Saved location

For the prototype, destinations outside the supported route should produce:

This route is currently unavailable.

26. Ride Search

After the passenger selects pickup and destination:

Finding available rides...

Checking:

- ✓ Route compatibility
- ✓ Pickup distance
- ✓ Destination compatibility
- ✓ Available seats
- ✓ Captain availability

27. Matching Results

Example:

Available Ride

Captain: Ahmad

Rating: 4.8 ★

Jerash → Amman

Pickup distance:

1.2 km

Available seats:

3

Estimated passenger fare:

3.00 JOD

[Request Ride]

Multiple matches can be displayed.

The ranking should prioritize the best match rather than simply the closest captain.

28. Ride Confirmation

Before confirming:

Trip

Pickup:

Jerash, Location X

Destination:

Amman, Location Y

Passengers:

1

Estimated fare:

3.00 JOD

Captain:

Ahmad

Available seats:

3

[Confirm Ride]

[Cancel]

29. Waiting Screen

After submitting:

Waiting for Captain

Ahmad

Toyota Corolla

License: XX-XXXX

ETA:

7 minutes

[Call]

[Message]

Trip status:

Captain accepted

30. Captain Tracking

Display:

- Captain location
- Passenger pickup location
- Route
- ETA
- Captain name
- Vehicle
- License plate

The captain's location should update in real time.

31. Trip In Progress

Display:

Trip in progress

Jerash → Amman

Captain:

Ahmad

Passengers:

3 / 4

Destination:

Amman

Estimated fare:

3.00 JOD

32. Trip Completion

After completion:

Trip Completed

Jerash → Amman

Fare:

3.00 JOD

Rate your captain

★★★★★

[Submit]

33. Captain UI Flow

Login



Captain Dashboard



Select Route



Configure Ride



Go Online



Receive Request



Accept



Navigate to Passenger



Pickup Passenger



Start Trip



Complete Trip

34. Captain Dashboard

Example:

Good morning, Ahmad

Status

● ONLINE

Current Route

Jerash → Amman

Available Seats

3 / 4

Pickup Radius

3 km

[Go Offline]

35. Route Configuration

Captain selects:

Origin

- Jerash

Destination

- Amman

Direction

- Jerash → Amman

or:

- Amman → Jerash

The system should only allow supported routes.

36. Pickup Radius

Captain selects acceptable pickup deviation.

Example:

Pickup Radius

1 km
2 km
3 km
5 km
10 km

Recommended default:

3 km

The radius should be configurable.

37. Available Seats

Maximum:

4 passengers

Captain can select:

1
2
3
4

The system automatically decreases available capacity as passengers are accepted.

38. Captain Ride Request

Example:

New Passenger Request

Passenger:

Mohammad

Pickup:

Jerash

Distance from route:

0.8 km

Destination:

Amman

Estimated detour:

1.4 km

Passengers:

1

[Accept]

[Reject]

39. Matching Engine

The Matching Engine is the core business component.

The system should not simply search for the nearest captain.

It should search for:

Captains whose predefined route is geographically compatible with the passenger's journey.

40. Route Model

Each captain has a route represented as a geographic polyline.

Example:

Jerash

|
|
|

Amman

Internally:

Route

└─ geometry: LINESTRING(...)

PostGIS stores this geometry.

41. Passenger Journey Model

The passenger journey contains:

pickupPoint

destinationPoint

Both are geographic points.

The matching system evaluates both against the captain's route.

42. Basic Matching Rules

A captain is eligible if:

Rule 1 — Same Direction

Passenger journey must correspond to the captain's direction.

Example:

Passenger:

Jerash → Amman

Captain:

Jerash → Amman

MATCH

But:

Passenger:

Jerash → Amman

Captain:

Amman → Jerash

should not be treated as the same direction.

Rule 2 — Available Seats

Captain must have enough seats.

If passenger requests:

2 seats

Captain must have:

≥ 2 available seats

Rule 3 — Pickup Distance

The passenger's pickup point must be within the captain's allowed pickup radius from the route.

Example:

Distance from route = 1.2 km

Captain radius = 3 km

MATCH

Rule 4 — Destination Compatibility

Passenger destination should also be sufficiently close to the captain's route.

Example:

Destination distance = 1.0 km

Maximum allowed = 3 km

MATCH

Rule 5 — Detour

The system should estimate whether serving the passenger creates an unacceptable detour.

This becomes increasingly important when multiple passengers are already onboard.

43. Matching Score

Each eligible captain receives a score.

Example:

Match Score =

Route Compatibility	40%
Pickup Distance	25%
Destination Distance	20%
Available Seats	10%
Detour	5%

The exact weights should be configurable.

44. Example Matching Calculation

Captain A:

Pickup distance: 0.7 km

Destination distance: 0.9 km

Seats: 3

Detour: 0.8 km

Score: 94%

Captain B:

Pickup distance: 1.8 km

Destination distance: 1.5 km

Seats: 2

Detour: 2.1 km

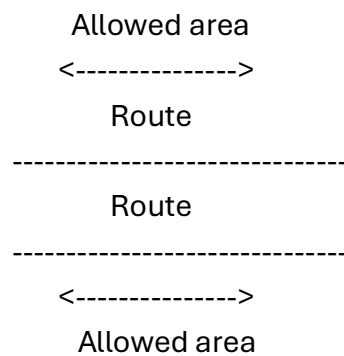
Score: 79%

Captain A should appear first.

45. Route Corridor

The prototype should conceptualize each route as a corridor.

Example:



If:

Captain pickup radius = 3 km

then the system considers passengers within approximately 3 km of the route.

This concept should later evolve into more sophisticated route deviation calculations.

46. Important Matching Constraint

The system must not match based purely on straight-line geographic distance.

Example:

Two locations may be geographically close but separated by roads that require a large detour.

Therefore:

Road distance and route compatibility should eventually take priority over simple Euclidean distance.

47. Seat Aggregation

Suppose a captain has:

Capacity = 4

Passenger A books:

1 seat

Remaining:

3

Passenger B books:

2 seats

Remaining:

1

Passenger C requests:

2 seats

Request must be rejected for this captain.

48. Trip Lifecycle

Trip statuses:

SEARCHING

MATCHED

REQUESTED

ACCEPTED

CAPTAIN_ARRIVING

PICKED_UP

IN_PROGRESS

COMPLETED

CANCELLED

State transitions must be controlled by the backend.

49. Database Design

Main entities:

users
captains
vehicles
routes
captain_routes
ride_requests
trips
trip_passengers
locations
ratings
notifications

50. Users Table

Example:

users

id
full_name
phone
email
password_hash
role
status
created_at
updated_at

Roles:

PASSENGER
CAPTAIN
ADMIN

51. Captains Table

captains

id

user_id

license_number

verification_status

rating

total_trips

created_at

updated_at

Verification statuses:

PENDING

APPROVED

REJECTED

SUSPENDED

52. Vehicles Table

vehicles

id

captain_id

make

model

year

color

license_plate

capacity
created_at
updated_at

The prototype capacity should not exceed 4 passengers.

53. Routes Table

routes

id
origin_city
destination_city
direction
geometry
status
created_at
updated_at

Example:

origin_city = Jerash
destination_city = Amman
direction = JERASH_TO_AMMAN

54. Captain Routes Table

captain_routes

id
captain_id

route_id
pickup_radius_km
available_seats
is_online
created_at
updated_at

55. Ride Requests Table

ride_requests

id
passenger_id
pickup_location
destination_location
requested_seats
status
created_at
updated_at

56. Trips Table

trips

id
captain_id
route_id
status
started_at
completed_at
created_at

updated_at

57. Trip Passengers Table

trip_passengers

id
trip_id
passenger_id
pickup_location
destination_location
seats
fare
status
created_at
updated_at

This table represents the relationship between passengers and trips.

58. Locations

The application should store location information carefully.

Possible model:

latitude
longitude
accuracy
timestamp

Real-time captain locations should preferably have a limited retention period to reduce unnecessary data storage.

59. Ratings

ratings

id

trip_id

from_user_id

to_user_id

rating

comment

created_at

Rating:

1-5

60. Backend Modules

NestJS modules:

auth

users

captains

vehicles

routes

ride-requests

matching

trips

locations

notifications
ratings
admin

61. Authentication Module

Responsibilities:

- Registration
- Login
- OTP
- Token generation
- Token refresh
- Logout
- Role validation

Authentication should use:

JWT access token + refresh token

62. User Registration API

POST /api/v1/auth/register

Example request:

```
{  
  "fullName": "Mohammad Abu Marar",  
  "phone": "+962XXXXXXXXX",  
  "role": "PASSENGER"  
}
```

63. Login API

POST /api/v1/auth/login

64. Request Ride API

POST /api/v1/ride-requests

Example:

```
{  
  "pickup": {  
    "latitude": 32.274,  
    "longitude": 35.896  
  },  
  "destination": {  
    "latitude": 31.95,  
    "longitude": 35.91  
  },  
  "requestedSeats": 1  
}
```

65. Matching API

GET /api/v1/ride-requests/{id}/matches

Response:

```
{
  "matches": [
    {
      "captainId": "123",
      "matchScore": 94,
      "pickupDistanceKm": 0.8,
      "destinationDistanceKm": 1.1,
      "availableSeats": 3,
      "estimatedFare": 3.0
    }
  ]
}
```

66. Captain Accept API

POST /api/v1/ride-requests/{id}/accept

67. Captain Reject API

POST /api/v1/ride-requests/{id}/reject

68. Start Trip API

POST /api/v1/trips/{id}/start

69. Complete Trip API

POST /api/v1/trips/{id}/complete

70. Cancel Trip API

POST /api/v1/trips/{id}/cancel

71. Location WebSocket

Example event:

captain.location.update

Payload:

```
{  
  "captainId": "123",  
  "latitude": 32.275,  
  "longitude": 35.897,  
  "timestamp": "2026-08-16T10:30:00Z"  
}
```

The backend broadcasts relevant updates to passengers associated with that trip.

72. WebSocket Events

Passenger:

ride.match.updated
ride.accepted
captain.location.updated
trip.started
trip.completed
trip.cancelled

Captain:

ride.request.created
ride.request.cancelled
passenger.joined
trip.updated

73. API Versioning

All APIs should begin with:

/api/v1/

Example:

/api/v1/auth/login
/api/v1/users/me
/api/v1/routes
/api/v1/trips

This allows future API versions without breaking existing clients.

74. Error Handling

Standard response format:

```
{
  "success": false,
  "error": {
    "code": "ROUTE_NOT_SUPPORTED",
    "message": "This route is currently unavailable."
  }
}
```

Potential error codes:

AUTH_INVALID
AUTH_EXPIRED
ROUTE_NOT_SUPPORTED
NO_MATCH_FOUND
NO_AVAILABLE_SEATS
CAPTAIN_OFFLINE
RIDE_ALREADY_ACCEPTED
RIDE_CANCELLED
INVALID_LOCATION

75. Security

The application must:

- Use HTTPS
- Hash passwords
- Never store plain-text passwords

- Validate JWT tokens
- Validate user roles
- Validate request ownership
- Rate-limit authentication endpoints
- Validate geographic input
- Protect captain personal information
- Avoid exposing unnecessary location information
- Sanitize user input

76. Location Privacy

Location data is sensitive.

The application should not continuously expose a captain's exact location to everyone.

Only relevant passengers should receive location updates during an active ride or pickup process.

Historical location data should have limited retention.

77. Permissions

Passenger:

- Location permission
- Notifications
- Phone permission if direct calling is supported

Captain:

- Location permission
- Background location where legally and technically required
- Notifications

Permissions must be requested contextually rather than immediately on application launch.

78. Pricing Prototype

Initial pricing can use a simple model.

Example:

Base Trip Price = 12 JOD

Then:

Passenger Fare =
Base Trip Price / Occupied Passenger Units

Example:

1 passenger = 12 JOD

2 passengers = 6 JOD

3 passengers = 4 JOD

4 passengers = 3 JOD

This is only a prototype pricing model.

79. Future Pricing Model

The production system may consider:

Base fare

+

Pickup deviation

+

Destination deviation

+

Distance

+

Passenger count

+

Time

However, pricing must remain predictable and understandable.

80. Ride Aggregation Strategy

The system should allow multiple passengers to join an existing trip if their journeys remain compatible.

Example:

Captain:

Jerash → Amman

Passenger A:

Jerash → Amman

Passenger B:

Jerash → Amman

Passenger C:

Jerash → University area, Amman

Passenger D:

Jerash → Downtown Amman

If all destinations remain within the route corridor, they can share the vehicle.

81. Dynamic Trip Matching

A captain does not necessarily need to wait for exactly four passengers before starting.

Possible states:

1/4 seats occupied

2/4 seats occupied

3/4 seats occupied

4/4 seats occupied

The captain may begin according to the business rules defined later.

82. Important Prototype Simplification

For the first prototype:

Do not implement complex multi-passenger route optimization.

Instead:

1. Captain defines route.
2. Passenger defines journey.
3. System checks compatibility.
4. Passenger joins.
5. Seat count decreases.

Once the basic model works, a more advanced algorithm can optimize multiple pickups and destinations.

83. Maps UX

The map should display:

- Current passenger location
- Pickup marker
- Destination marker
- Captain location
- Route
- Pickup radius where appropriate

Markers should have clear semantic meaning.

84. Loading States

Every network-dependent screen needs a loading state.

Examples:

Searching for rides...

Loading map...

Finding captains...

Accepting ride...

Updating trip...

Never leave the user looking at a frozen screen.

85. Empty States

Example:

No rides available

We couldn't find a captain matching
your route right now.

Try again or check back shortly.

CTA:

Search Again

86. Error States

Example:

Location unavailable

Please enable location services
to select your pickup point.

[Enable Location]

87. Offline Behavior

If the passenger loses internet:

Display:

Connection lost

Trying to reconnect...

The application should automatically attempt reconnection.

Critical trip state must always be confirmed by the backend.

88. Notification Strategy

Example notification:

Captain Accepted

Ahmad has accepted your ride request.

Example:

Captain Nearby

Your captain is approximately 500 meters away.

Example:

Trip Started

Your trip to Amman has started.

89. Admin Dashboard

Main sections:

Dashboard

Users

Captains

Vehicles

Routes

Active Trips

Ride Requests

Complaints
Ratings
System Settings

90. Admin Dashboard Statistics

Example:

Today's Trips
125

Active Captains
42

Active Passengers
178

Completed Trips
98

Cancelled Trips
12

Average Rating
4.7

91. Route Management

Admin can:

- Create route

- Edit route
- Enable/disable route
- Set route geometry
- Set maximum radius
- Set pricing configuration

Example:

Jerash → Amman

Status: ACTIVE

Amman → Jerash

Status: ACTIVE

92. Testing Strategy

The prototype must have:

Unit Tests

Test:

- Matching calculations
- Seat calculations
- Pricing
- Authentication
- Trip state transitions

Integration Tests

Test:

Passenger request

→ Matching

→ Captain acceptance

→ Trip creation

API Tests

Use Postman or automated API tests.

Mobile Tests

Test:

- Android
- iOS
- Different screen sizes
- GPS permission states
- Network loss

93. Critical Matching Test Cases

Test 1

Captain route:

Jerash → Amman

Passenger:

Jerash → Amman

Expected:

MATCH

Test 2

Captain:

Amman → Jerash

Passenger:

Jerash → Amman

Expected:

NO MATCH

Test 3

Pickup outside radius.

Expected:

NO MATCH

Test 4

Destination outside route corridor.

Expected:

NO MATCH

Test 5

Captain has zero available seats.

Expected:

NO MATCH

Test 6

Passenger requests 2 seats, captain has 1.

Expected:

NO MATCH

Test 7

Two passengers join a four-seat captain.

Expected:

Available seats = 2

94. Performance Requirements

The matching operation should ideally return results quickly.

Target prototype:

< 2 seconds

under normal conditions.

Real-time location updates should not unnecessarily overload the backend.

Location update frequency can initially be approximately:

every 3–5 seconds during active trips

and reduced when the captain is not actively serving a passenger.

95. Scalability Considerations

The architecture should allow future introduction of:

- Redis
- Message queues
- Background workers
- Load balancing
- Multiple backend instances

For the prototype, these are optional.

A simple deployment is sufficient.

96. Suggested Future Redis Usage

Redis can eventually handle:

- Online captain state
- Real-time location cache
- Matching cache
- WebSocket scaling
- Temporary trip state

Example:

PostgreSQL

= permanent data

Redis

= temporary real-time data

97. Deployment Architecture

Prototype:

Flutter Apps

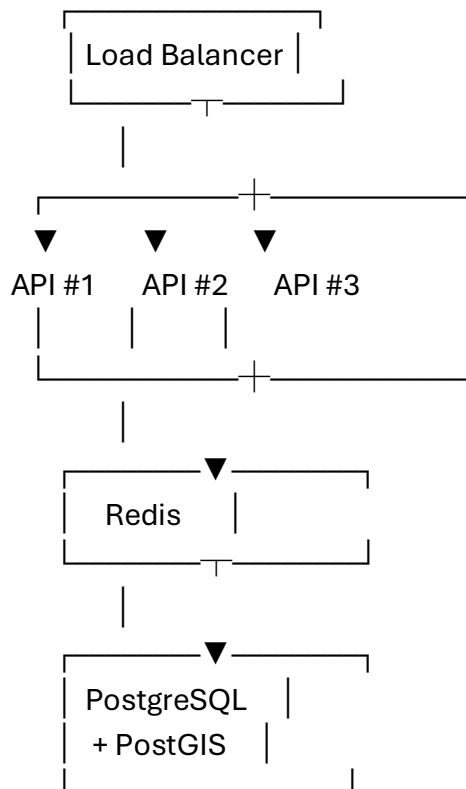


Cloud Backend



PostgreSQL

Production:



98. Environment Configuration

Use separate environments:

Development
Staging
Production

Example:

.env.development
.env.staging
.env.production

Secrets must never be committed to Git.

99. Git Branching Strategy

Recommended:

main
develop
feature/*
bugfix/*

Example:

feature/passenger-home
feature/matching-engine
feature/captain-dashboard
feature/location-tracking

100. CI/CD

Later, use:

- GitHub Actions
- Automated tests
- Build verification
- Flutter builds
- Backend deployment

Prototype does not require a complicated CI/CD system, but the repository should be structured for it.

101. Repository Structure

Recommended monorepo:

```
tawsileh/
|
|— mobile/
|   └─ Flutter application
|
|— backend/
|   └─ NestJS application
|
|— admin/
|   └─ Next.js dashboard
|
|— docs/
|   └─ architecture/
|       └─ api/
|           └─ database/
|               └─ ui/
```

```
|— docker/  
|  
└— README.md
```

102. Design System

Define reusable:

Colors

Use a primary brand color.

Example conceptual palette:

Primary
Secondary
Background
Surface
Text
Success
Warning
Error

Exact colors should be finalized in Figma.

103. Typography

Use a font that supports Arabic and English.

The design system should define:

Display
Heading 1
Heading 2
Heading 3
Body
Caption
Button

104. UI Components

Create reusable components:

PrimaryButton
SecondaryButton
LocationPicker
MapView
RideCard
CaptainCard
TripStatusCard
PriceCard
SeatSelector
RouteSelector
BottomSheet
RatingWidget
LoadingIndicator
ErrorState
EmptyState

105. Passenger Bottom Navigation

Recommended:

Home
Trips
Notifications
Profile

106. Captain Bottom Navigation

Recommended:

Dashboard
Trips
Notifications
Profile

The captain dashboard should emphasize:

Online / Offline status

107. Accessibility

The application should support:

- Large touch targets
- Readable text
- High contrast
- Screen reader labels
- Clear error messages
- Non-color-only status indicators

108. Localization

All strings must be externalized.

Do not hardcode:

```
Text("Find a Ride")
```

Instead use localization:

```
Text(AppLocalizations.of(context)!.findRide)
```

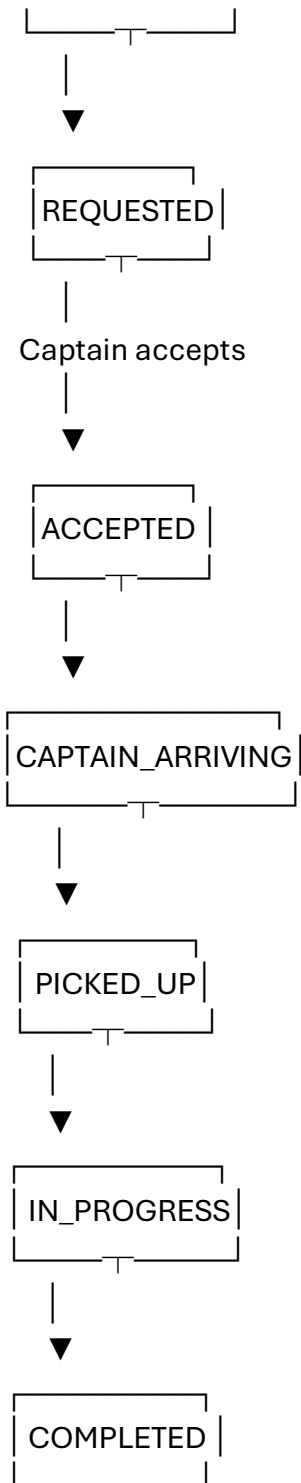
This makes Arabic/English support easier.

109. Core Business Rules

1. Maximum passenger capacity is 4.
2. Captain must select a supported route.
3. Captain must be online to receive requests.
4. Captain must have available seats.
5. Passenger must provide pickup and destination.
6. Passenger journey must match captain direction.
7. Pickup must be within allowed route radius.
8. Destination must be compatible with route.
9. Backend is authoritative for trip state.
10. Client applications must never independently determine final trip status.

110. Prototype State Machine

SEARCHING



Cancellation can occur at appropriate states.

111. Analytics Events

The prototype should optionally record events such as:

app_opened
registration_completed
location_selected
destination_selected
ride_search_started
match_found
ride_requested
ride_accepted
ride_cancelled
trip_started
trip_completed
rating_submitted

These events will later help determine where users abandon the process.

112. Product Metrics

Important future metrics:

Matching Success Rate

Successful Matches / Total Requests

Average Match Time

How long passengers wait before receiving a suitable match.

Captain Utilization

Percentage of available seats actually occupied.

Average Occupancy

Example:

Average = 3.1 passengers / vehicle

Cancellation Rate

Cancelled rides / Requested rides

Average Trip Cost

Average amount paid by each passenger.

113. Prototype Success Criteria

The prototype is successful if:

1. Passenger can register.
2. Captain can register.
3. Captain can define Jerash → Amman.
4. Passenger can select pickup and destination.
5. System correctly identifies compatible captains.
6. System respects pickup radius.
7. System respects destination compatibility.
8. System respects seat availability.
9. Captain receives request.
10. Captain can accept.
11. Passenger receives confirmation.
12. Captain location can be displayed.

13. Trip can start.
14. Trip can complete.
15. Seats are updated.
16. Trip appears in history.

114. Development Phases

Phase 1 — Project Setup

- Create Git repository
- Create Flutter project
- Create NestJS project
- Create PostgreSQL database
- Configure PostGIS
- Configure environments
- Create base architecture

Phase 2 — UI/UX

Create Figma screens:

- Splash
- Onboarding
- Login
- Passenger home
- Pickup selection
- Destination selection
- Matching
- Ride confirmation
- Captain dashboard
- Route selection
- Ride request

- Active trip
- Profile
- Trip history

Phase 3 — Authentication

Implement:

- Registration
- Login
- OTP
- JWT
- User roles

Phase 4 — Maps

Implement:

- Map rendering
- Current location
- Search
- Pickup marker
- Destination marker
- Route drawing

Phase 5 — Captain System

Implement:

- Captain profile
- Vehicle
- Route

- Radius
- Available seats
- Online/offline status

Phase 6 — Matching Engine

Implement:

Route filtering

↓

Direction validation

↓

Seat filtering

↓

Pickup distance

↓

Destination compatibility

↓

Detour estimation

↓

Match score

↓

Ranking

Phase 7 — Ride Lifecycle

Implement:

Request

→ Accept

→ Arrive

→ Pickup

→ Start

→ Complete

Phase 8 — Real-Time

Implement:

- WebSocket
- Captain location
- Trip updates
- Ride request events

Phase 9 — Notifications

Implement FCM notifications.

Phase 10 — Admin

Build basic admin dashboard.

115. Recommended First Milestone

Do NOT build everything immediately.

The first technical milestone should be:

Passenger selects a location in Jerash and destination in Amman, while a fake captain with a predefined Jerash → Amman route exists in the database. The backend determines whether the passenger matches the captain based on route and radius and displays the result in the Flutter application.

This milestone validates the most important concept in the entire product.

116. Second Milestone

Add:

Real Captain
+
Real Passenger
+
Ride Request
+
Captain Acceptance

117. Third Milestone

Add:

Real-Time Location
+
Trip Lifecycle
+
Notifications

At this point, the prototype becomes a functional end-to-end transportation platform.

118. Fourth Milestone

Add:

Multiple Captains
+
Multiple Passengers
+
Seat Aggregation
+
Dynamic Matching

This is where the shared-ride concept becomes fully demonstrable.

119. Future Expansion

Once Jerash ↔ Amman is validated, the database should allow new routes without changing application logic.

Example:

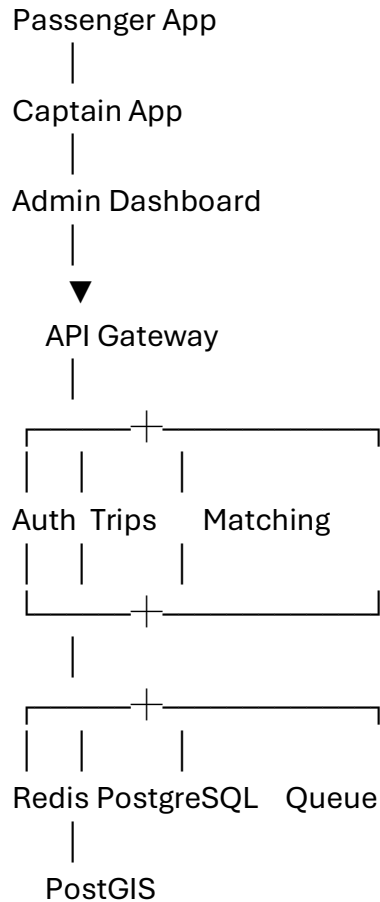
routes

- 1 | Jerash → Amman
- 2 | Amman → Jerash
- 3 | Irbid → Amman
- 4 | Amman → Irbid
- 5 | Zarqa → Amman
- 6 | Amman → Zarqa

The application should retrieve active routes dynamically.

120. Long-Term Architecture

Eventually:



Potential future services:

- Payment Service
- Notification Service
- Pricing Service
- Routing Service
- Fraud Detection
- Analytics Service

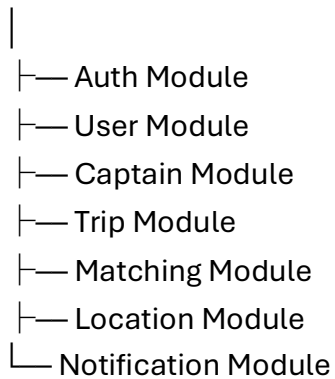
These should not be separate microservices during the initial prototype.

121. Important Engineering Principle

The prototype should use a **modular monolith**, not microservices.

Recommended:

One Backend



This provides clean architecture without unnecessary operational complexity.

Microservices can be introduced only if the system eventually requires them.

122. Recommended Final Prototype Stack

Mobile

Flutter + Dart

Backend

Node.js + NestJS + TypeScript

Database

PostgreSQL + PostGIS

Real-Time

WebSocket + Socket.IO

Maps

Google Maps Platform

Notifications

Firebase Cloud Messaging

Admin

Next.js + React + TypeScript

Design

Figma

Testing

Postman + Jest + Flutter Test

DevOps

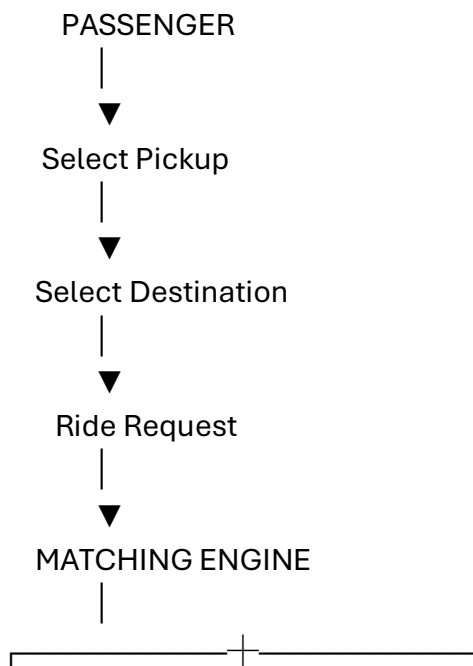
Docker + GitHub Actions

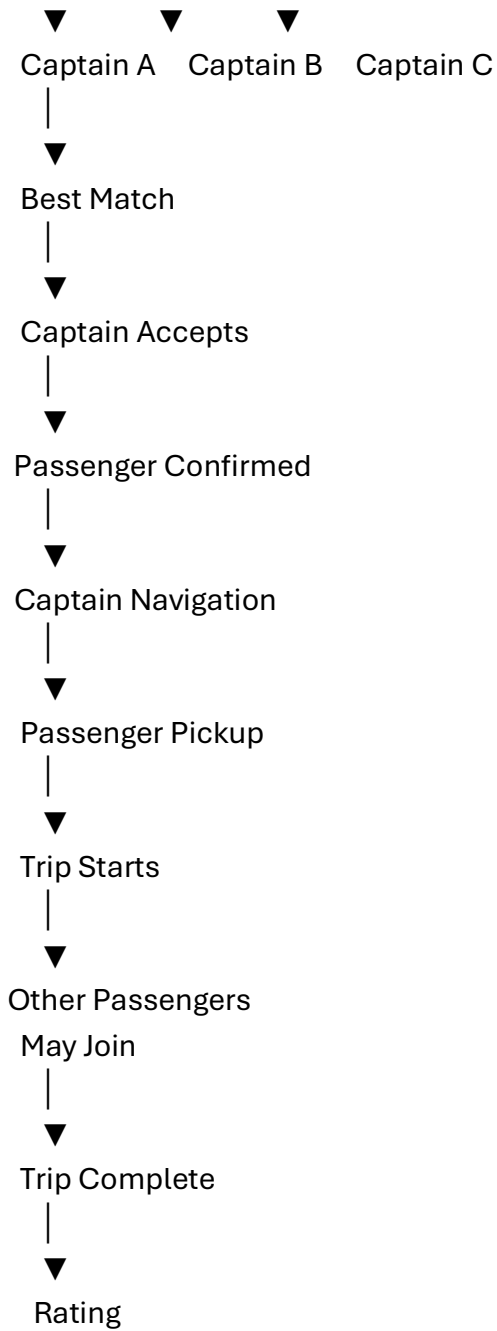
Version Control

Git + GitHub

123. Final Product Flow

The complete prototype should demonstrate:





124. Definition of Done

The Tawsileh prototype can be considered complete when a real user can perform the following without developer intervention:

Passenger

- Register
- Log in
- Allow location access
- Select Jerash pickup
- Select Amman destination
- Search for a ride
- Receive a matching captain
- Submit request
- Receive acceptance
- Track captain
- Complete trip
- Rate captain

Captain

- Register
- Configure vehicle
- Select Jerash → Amman
- Set pickup radius
- Set available seats
- Go online
- Receive passenger request
- Accept passenger
- Navigate to pickup
- Start trip
- Complete trip

Backend

- Authenticate users
- Store users
- Store captains
- Store routes
- Store geographic data
- Perform route matching

- Calculate compatibility
- Manage seats
- Manage trip states
- Broadcast real-time events
- Send notifications

125. Final Recommendation

The most important part of Tawsileh is not the map UI, login system, or even the mobile application itself.

The **core intellectual/technical component** is the:

Route-Based Passenger Matching Engine

The architecture should therefore be designed around:

Passenger Journey

+

Captain Route

+

Geographic Compatibility

+

Available Capacity

+

Detour

↓

MATCH SCORE

The first version should remain intentionally simple.

The goal is not to build a complete Uber competitor.

The goal is to prove one thing:

Can Tawsileh reliably find multiple passengers and captains traveling along compatible intercity routes and combine them into a shared trip while reducing the cost per passenger?

If the answer is yes, the same architecture can subsequently be expanded from:

Jerash ↔ Amman

to a nationwide intercity transportation network.